

Digital Logic Simulator

Akash Somasundaram ee24b084

October 2025

1 Introduction

The program implements a minimal combinational logic simulator capable of parsing, validating, and evaluating digital circuits defined in a structured text format. The goal is to simulate combinational behavior for a given circuit description and output its signal waveforms in **WaveDrom JSON** format.

2 Functions Implemented

2.1 `parse_netlist(text)`

Parses the `.net` input file containing sections for `INPUTS:`, `OUTPUTS:`, `GATES:`, and `STIMULUS:`.

Validates correct ordering and structure of each section, ensuring proper gate definitions and well-formed stimuli.

Builds a `Circuit` object that stores all parsed information for later simulations.

2.2 `eval_gate`

Evaluates the logic of a single gate. Supports **AND**, **OR**, **XOR** (two inputs each), and **NOT** (single input). Ensures correct input counts and returns the logical output (0 or 1).

2.2.1 `simulate`

Performs the actual circuit simulation. For each stimulus line, it assigns input values, evaluates all gates in order, and records resulting signal states for every wire. Produces a dictionary mapping each signal to its time-varying values.

2.3 `make_wavedrom_json`

Converts the simulation output into **WaveDrom-compatible JSON** for wave-form visualisation. Each signal is represented by its name and binary wave sequence.

3 Evaluation Strategy

To make the simulator independent of gate ordering, an iterative dependency-resolution approach is used.

For each stimulus step, all input values are initialized first. The simulator then repeatedly scans through all gate definitions, evaluating any gate whose input signals are already known. Each successfully evaluated gate adds a new signal to the current state, and the process repeats until all gates are resolved.

If a full pass makes no progress, the program reports an error indicating a cyclic or undefined dependency.

This method naturally produces the correct evaluation order without requiring an explicit topological sort and guarantees correct results for any valid combinational circuit.

4 How I worked and resources

The entire simulator was implemented using only Python's standard library modules, including `re`, `json`, `argparse`, and `pathlib`. I designed and developed both the parser and the logic simulator from first principles, focusing on clear structure and minimal dependencies. During development, I used ChatGPT to discuss design approaches, clarify implementation details, and verify the correctness of my evaluation strategy. All code was written, tested, and fully understood by me, with ChatGPT serving only as a programming assistant.

Running the simulator

From a shell in the folder containing `digitalsim.py` and a netlist file (e.g. `and2.net`) run:

```
python digitalsim.py and2.net
```

The simulator writes `and2.json` by default; specify `-o out.json` to change the output path.